Lab2-DDPM

Li Ruei Bin
Student ID: 314552050

Hyperparameter

1. SDS

–guidance_scale 25 –lr 0.01 –steps 1000

2. VSD

–guidance_scale 7.5 – 1e-4 –lora_loss_weight 1.0 –lora_rank 4 –steps 1000

3. SDI

–lr 0.005 –steps 1000 –guidance_scale 7.5 –inversion_n_steps 10 –inversion_guidance_scale -7.5 –inversion_eta 0.0 –sdi_update_interval 25

Code

1. TODO1 - get_sds_loss:

Divide the ddim equation to three parts.

a. Sample a timestep t.

b. Sample a random noise.

c. Add noise to the latents to get latents_noisy.

d. Use latents_noise, t, text_embedding and guidance noise to get predicted_noise.

e. Get w(t) first and then use it to calculate gradient.

f. Use gradient and latents to get loss and return it.

```
def get_sds_loss(self, latents, text_embeddings, guidance_scale=7.5):
    """
    Score Distillation Sampling (SDS) Loss

    Reference: DreamFusion (https://arxiv.org/abs/2209.14988)
    """
    # TODO: Implement SDS loss
    t = (
        torch.randint(self.min_step, self.max_step + 1, size=(latents.shape
            [0],))
        .to(latents.device)
        .long()
    )
    noise = torch.randn_like(latents)

    latents_noisy = self.scheduler.add_noise(latents, noise, t)
    noise_pred = self.get_noise_preds(latents_noisy, t, text_embeddings,
        guidance_scale)
    w_t = (1 - self.alphas[t]).reshape(-1, 1, 1, 1)
```

```
18
19      grad = w_t * (noise_pred - noise)
20      loss = (grad.detach() * latents).mean()
21      return loss
```

Listing 1: Get SDS Loss

2. TODO2 - get_vsd_loss:

a. Sample t, noise and get latents_noisy first.

b. First, get lora_noise_pred through the base model plus lora.

c. Second, get base_noise_pred by base model. I turn off the lora adapters by self.unet.enable_adapters(). Then, turn on it by self.unet_enable_adapters()

d. Use w(t), lora_pred_noise and base_pred_noise to get the gradient of vsd, and then use the gradient to get loss of vsd.

e. Detach the latents_noisy and use it to calculate the lora_noise_pred_for_loss.

f. Calculate lora_loss by lora_noise_pred_for_loss and noise

g. The final loss is (lora_loss * lora_loss_weights) + loss_vsd

```python
def get_vsd_loss(self, latents, text_embeddings, guidance_scale=7.5, lora_loss_weight=1.0):
    """
    Variational Score Distillation (VSD) Loss

    Reference: ProlificDreamer (https://arxiv.org/abs/2305.16213)
    """
    # TODO: Implement VSD loss
    t = (
        torch.randint(self.min_step, self.max_step + 1, size=(latents.shape[0],))
        .to(latents.device)
        .long()
    )
    noise = torch.randn_like(latents)
    latents_noisy = self.scheduler.add_noise(latents, noise, t)

    self.unet.enable_adapters()
    lora_noise_pred = self.get_noise_preds(latents_noisy, t, text_embeddings, guidance_scale)

    self.unet.disable_adapters()
    base_noise_pred = self.get_noise_preds(latents_noisy, t, text_embeddings, guidance_scale)
    self.unet.enable_adapters()

    w_t = (1 - self.alphas[t]).reshape(-1, 1, 1, 1)

    grad_vsd = w_t * (base_noise_pred - lora_noise_pred)
    loss_vsd = (grad_vsd.detach() * latents).mean()

    latents_noisy_detached = latents_noisy.detach()
    lora_noise_pred_for_loss = self.get_noise_preds(latents_noisy_detached, t, text_embeddings, guidance_scale)

    lora_loss = F.mse_loss(lora_noise_pred_for_loss, noise.detach())

    return (lora_loss * lora_loss_weight) + loss_vsd
```

Listing 2: Get VSD Loss

TODO3 - invert_noise:

a. The initial target_t is tensor, so convert it to scalar first.

b. Use the scalar to initial the timesteps.

c. Set latents as x_t.

d. Use n_steps to iterate the for-loop.

e. Inside the for-loop, first get t_current and t_next.

f. Second, convert current_t to a tensor and use it to get noise_pred.

g. Third, calculate the $x_{\mathbf{t+1}}$ by formula and call it xt_next_deterministic.

h. Finally, if $\eta > 0$, we need variance to get the final $x_{\mathbf{t+1}}$ or $x_{\mathbf{t+1}}$ = xt_next_deterministic.

```python
@torch.no_grad()
def invert_noise(self, latents, target_t, text_embeddings, guidance_scale=-7.5, n_steps=10, eta=0.3):
    """
    DDIM Inversion: x0 -> x_t

    Inverts clean latents (x0) to noisy latents (x_t) using DDIM inversion.

    Args:
        latents: Clean latents x0
        target_t: Target timestep to invert to
        text_embeddings: Text condition
        guidance_scale: CFG scale (typically negative for inversion!)
        n_steps: Number of inversion steps
        eta: Noise level for stochasticity

    Returns:
        Inverted noisy latents x_t
    """
    # TODO: (Implement DDIM inversion by yourself - do NOT call built-in inversion helpers):
    # ----------------------------------------------------------------------
    # Write your own DDIM inversion loop that maps x0 -> x_t at `target_t`.
    # You may *read* external implementations for reference, but you must
    # NOT call any "invert"/"ddim_invert"/"invert_step" utilities
    # from diffusers or other libraries.
    if torch.is_tensor(target_t) and target_t.dim() > 0:
        target_t_scalar = target_t[0].item()
    else:
        target_t_scalar = target_t

    timesteps = torch.linspace(0, target_t_scalar, n_steps + 1).long().to(latents.device)
    xt = latents

    for i in range(n_steps):
        t_current = timesteps[i]
        t_next = timesteps[i + 1]

        if t_current == t_next:
            continue

        t_current_tensor = torch.full((latents.shape[0],), t_current, device=latents.device).long()
        noise_pred = self.get_noise_preds(xt, t_current_tensor, text_embeddings, guidance_scale)

```

```
        alpha_t = self.scheduler.alphas_cumprod[t_current]
        alpha_t_next = self.scheduler.alphas_cumprod[t_next]
        first_part = torch.sqrt(alpha_t_next / alpha_t) * xt
        second_part = (torch.sqrt(1.0 / alpha_t_next - 1.0) - torch.sqrt(1.0 /
            alpha_t - 1)) * torch.sqrt(alpha_t_next) * noise_pred
        xt_next_deterministic = first_part + second_part

        if eta > 0:
            variance = ((1.0 - alpha_t) / (1.0 - alpha_t_next)) * (1.0 -
                alpha_t_next / alpha_t)
            variance = torch.clamp(variance, min=1e-20)
            std_dev_t = eta * torch.sqrt(variance)
            noise = torch.randn_like(xt)
            xt_next = xt_next_deterministic + std_dev_t * noise
        else:
            xt_next = xt_next_deterministic

        xt = xt_next
    return xt
```

Listing 3: Invert Noise

TODO4 - get_sdi_loss

a. Get current t.

b. Get noise_pred

c. Use alpha_t, latents_noisy and noise_pred to calculate the target x_0

d. Calculate the final loss by MSE Loss and return it.

```python
def get_sdi_loss(
    self,
    latents,
    text_embeddings,
    guidance_scale=7.5,
    current_iter=0,
    total_iters=500,
    inversion_guidance_scale=-7.5,
    inversion_n_steps=10,
    inversion_eta=0.3,
    update_interval=25,
):
    """
    Score Distillation via Inversion (SDI) Loss

    Reference: Score Distillation via Reparametrized DDIM (https://arxiv.org/abs/2405.15891)

    Key Insight: Instead of using random noise like SDS, SDI uses DDIM inversion
    to get better noise that's consistent with the current latents.

    Strategy:
    1. Timestep annealing: t decreases from max_step to min_step during training
    2. Periodically update target via DDIM inversion
    3. Use MSE loss between current latents and cached target

    Args:
        latents: Current optimized latents (B, 4, H, W)
        text_embeddings: Concatenated [uncond, cond] embeddings (2*B, seq_len, dim)
        guidance_scale: CFG scale for final denoising step
        current_iter: Current training iteration number
        total_iters: Total number of training iterations
        inversion_guidance_scale: CFG scale for DDIM inversion (typically negative)
        inversion_n_steps: Number of inversion steps from x0 to x_t
        inversion_eta: Stochasticity level for DDIM inversion (0=deterministic)
        update_interval: Update cached target every N iterations

    Returns:
        loss: MSE loss between current latents and cached target
    """
    B = latents.shape[0]

    # TODO: Create current timestep tensor based on training progress
    progress = current_iter / total_iters
    t_scalar = self.max_step - (self.max_step - self.min_step) * progress
    t_scalar_int = int(t_scalar)

    t = torch.full((B,), t_scalar_int, device=latents.device, dtype=torch.long)

    # Check if we need to update target
```

```
    should_update = (current_iter % update_interval == 0) or not hasattr(self,
        'sdi_target')

    if should_update:
        with torch.no_grad():
            # Perform DDIM inversion: x0 -> x_t
            latents_noisy = self.invert_noise(
                latents, t, text_embeddings,
                guidance_scale=inversion_guidance_scale,
                n_steps=inversion_n_steps,
                eta=inversion_eta
            )

            # TODO: Predict noise from inverted noisy latents
            noise_pred = self.get_noise_preds(latents_noisy, t, text_embeddings
                , guidance_scale)

            # TODO: Denoise to get target x0 using predicted noise
            alpha_t = self.alphas[t].reshape(-1, 1, 1, 1)
            target = (latents_noisy - torch.sqrt(1.0 - alpha_t) * noise_pred) /
                 torch.sqrt(alpha_t)

            # Cache the target
            self.sdi_target = target.detach()

    # TODO: Compute MSE loss between current latents and cached target
    loss = F.mse_loss(latents, self.sdi_target)

    return loss
```

Listing 4: Get SDI Loss

Evaluation

Image

SDS ——— VSD ——— SDI

1. A red bus driving on a desert road

2. a boat in a river

3. A cabin surrounded by forests

4. A church beside a lake

5. A villa close to the pool

6. A castle next to a river

7. A burger on the table

8. A dog sitting on grass

9. a cat sitting on a table

10. A car on the road

CLIP

1. SDS: 0.3031
2. VSD: 0.3052
3. SDI: 0.3002

Question

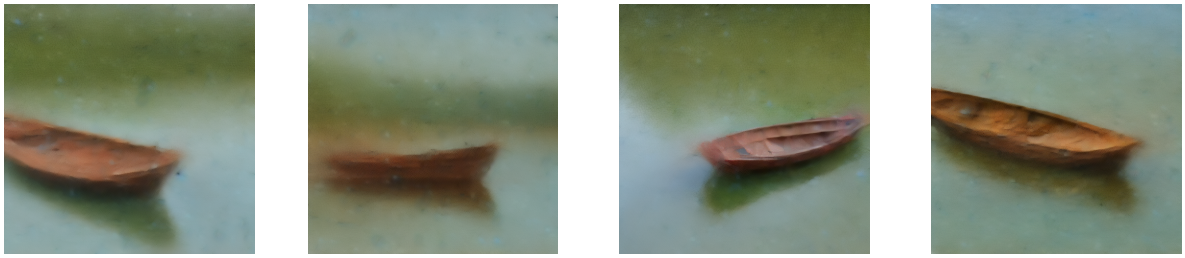
Question: Intuitively analyze why SDI and VSD perform better than SDS (without heavy math).

Answer: When denoise, SDS go random steps to get clean image. SDI always knows where is the noise from, so it can denoise the image cleaner. In VSD, the difference between the teacher (base model) and student (lora) is cleaner, so it can learn the noise better.

Ablation Study

SDS: guidance scale

The guidance in 25 is the blurriest. Smaller and bigger values are better.

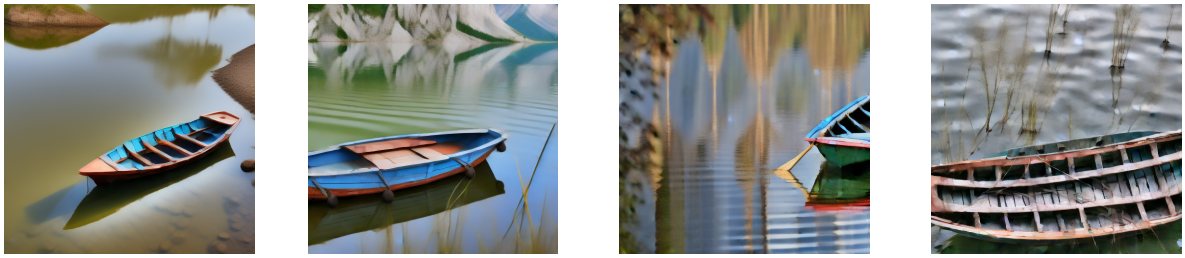


(a) 22.5 (b) 25 (c) 27.5 (d) 30

Figure 1: This is the prompt (a boat in a river) for all four images with difference guidance in SDS.

SDI: inversion steps

The higher the value, the more realistic the images are, but the images looks more unclear.



(a) 10 (b) 20 (c) 50 (d) 100

Figure 2: This is the prompt (a boat in a river) for all five images with difference inversion steps in SDI.

SDI: update interval

The smaller the value, the more realistic the images are. The image looks the most clear in the value 25.

(a) 15 (b) 20 (c) 25 (d) 30 (e) 35

Figure 3: This is the prompt (a boat in a river) for all five images with difference update interval in SDI.